AI-POWERED CUSTOMER SUPPORT TICKETING SYSTEM

A Salesforce-Based Intelligent Case Management Solution

Project Report

Submitted as part of the Salesforce Developer Program

Prepared by

Durga Surya Teja,

suryatejadurga2@gmail.com

Platform: Salesforce Developer Edition

Technologies: Flow Builder · Agentforce · Prompt Builder · Reports & Dashboards

Table of Contents

User Story

As a support agent using the AI-Powered Customer Support Ticketing System, I want to log, classify, and resolve customer issues efficiently so that customers receive timely and accurate help without unnecessary delays.

I want the system to automatically classify incoming cases by issue type, assign the correct priority, and calculate the SLA due date, so that no case is misrouted or left unattended beyond its committed resolution window.

I want an AI assistant that can read the case details and suggest a professional, ready-to-send reply, so that I can respond to customers faster while still reviewing and personalizing the message before it goes out.

I want to record whether the AI's suggestion was actually helpful, so that the system's effectiveness can be tracked and improved over time.

As a support manager, I want visibility into open cases, SLA compliance, and agent performance through reports and dashboards, so that I can identify bottlenecks and coach my team proactively.

As a system administrator, I want role-based access control through profiles and permission sets, so that agents, managers, and administrators each see and edit only what is relevant to their role, keeping customer data secure.

1. Introduction

1.1 Project Overview

The "AI-Powered Customer Support Ticketing System" is a Salesforce-based case management solution built to help customer support teams manage, classify, and resolve customer issues more efficiently. The system is implemented entirely using core Salesforce Administration, Flow Builder, and Agentforce capabilities within a Salesforce Developer Org, without relying on external AI services or custom Apex code.

At its core, the application centralizes customer support operations around the standard Case object, extended with custom objects for Support Category, SLA Policy, and AI Feedback. Record-triggered flows automatically classify incoming cases, assign priority, and calculate SLA due dates the moment a case is created or updated. An Agentforce-powered assistant, built using Prompt Builder and a custom Agent Topic, reads the case details and generates a suggested, professional reply that agents can review, personalize, and send — significantly reducing response time while keeping a human in the loop.

Role-based security, implemented through cloned profiles and dedicated permission sets, ensures that support agents, support managers, and administrators each have access appropriate to their responsibilities. Reports and dashboards give managers real-time visibility into case volume, SLA compliance, and individual agent performance, while an AI feedback loop captures whether agents found the AI's suggestions useful — creating a foundation for continuous improvement.

1.2 Purpose

The purpose of this project is to:

- Centralize and standardize how customer support cases are logged, tracked, and resolved.
- Automate repetitive administrative work — case classification, priority assignment, and SLA calculation — that would otherwise be done manually by agents.
- Assist agents with AI-generated response suggestions so replies are faster, more consistent, and more professional.
- Provide support managers with real-time, data-driven visibility into team performance and SLA compliance.
- Protect customer and case data through role-based access control, field-level security, and validation rules.
- Establish a feedback mechanism that measures whether AI-generated suggestions are genuinely useful to agents, enabling continuous refinement of the system.

1.3 Scope of the Document

This document covers the complete lifecycle of the project — from requirement gathering and stakeholder mapping, through system design, data modeling, automation, AI integration, and security configuration, to testing, reporting, and future scope. Screenshots of the actual implementation in the Salesforce Developer Org are included throughout to demonstrate that the described functionality has been built and verified.

2. Ideation Phase

2.1 Problem Statement

Customer support teams that rely on manual processes face recurring, predictable pain points. The table below captures these pain points in a structured customer-problem-statement format, identifying who is affected, what they are trying to do, what is blocking them, why the blocker exists, and how it makes them feel.

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	Support Agent	Resolve customer cases quickly and correctly	Cases are not classified or prioritized consistently	Classification and priority are assigned manually	Overwhelmed and inefficient
PS-[1]	Support Manager	Monitor SLA compliance and team performance	Reports are delayed and scattered across records	There is no centralized reporting on case data	Uncertain and reactive
PS-3	Customer	Get a fast, accurate resolution to my issue	Responses are slow and inconsistent in quality	Agents draft every reply from scratch	Frustrated and undervalued

Stakeholder Group	Role in the System
Customers	Raise support requests and receive suggested-reply based resolutions and status updates.
Support Agents	Handle and resolve cases, review AI-suggested replies, and record feedback on suggestion quality.
Support Managers	Monitor SLA compliance and agent performance using reports and dashboards.
System Administrators	Configure objects, automation, security, and Agentforce settings; maintain the system.
Business Stakeholders	Define requirements and validate that the solution aligns with organizational support goals.

[1] .2 Stakeholder Mapping

Stakeholder mapping identifies the key individuals and groups involved in the development, implementation, and day-to-day use of the system. Each stakeholder group has distinct responsibilities and interests in the project's success.

2.3 Brainstorming

Before settling on the final approach, several alternative solutions were considered for automating customer support case handling:

- Continue with a fully manual, spreadsheet-based ticket log.
- Build a simple email-based ticketing workflow with no classification logic.
- Use a third-party helpdesk tool with a paid AI add-on.
- Build a Salesforce-native ticketing system using standard objects and Flow automation only, without AI.
- Build a Salesforce-native ticketing system enhanced with Agentforce and Prompt Builder for AI-assisted classification and reply suggestions (selected idea).

Selected Idea: A Salesforce-based AI-Powered Customer Support Ticketing System, built on core Admin and Flow Builder features and enhanced with Agentforce for AI-assisted case handling — chosen for its low implementation cost, strong scalability, native security model, and ability to demonstrate real-world Salesforce and Agentforce skills.

3. Requirement Analysis Phase

3.1 Requirement Gathering & Planning

The requirement gathering phase focused on understanding the challenges faced by customer support teams in handling support requests efficiently. Stakeholders required a centralized system to manage customer issues, reduce manual effort in case classification, ensure timely resolution through SLA tracking, and improve agent productivity. The system needed to automatically categorize cases, assign appropriate priority, assist agents with suggested actions, and provide visibility into support performance through reports.

Based on these discussions, functional and non-functional requirements were identified, emphasizing ease of use, automation, data accuracy, role-based access control, and scalability. The solution was designed to be implemented using core Salesforce features in a Developer Org, along with Agentforce and Prompt Builder for AI capabilities, ensuring simplicity, reliability, and ease of maintenance.

3.2 Functional Scope

The functional scope of the project defines the key features and capabilities delivered by the Salesforce-based customer support ticketing system. The application allows users to create and manage customer support cases, automatically classify cases based on issue type, assign priorities, and calculate SLA due dates.

Support agents can view AI-suggested replies to resolve cases, update case statuses, and communicate with customers. The system also captures agent feedback on automated suggestions to evaluate system effectiveness. Role-based access control ensures that support agents, support managers, and administrators have appropriate permissions. Managers can monitor support performance using reports, while administrators can configure automation, security, and data models.

Functional Requirements

FR No.	Requirement
FR-1	Case creation and management with automatic classification
FR-2	Automatic priority assignment based on issue type
FR-3	Automatic SLA due date calculation
FR-4	AI-generated suggested reply for each case using Agentforce and Prompt Builder
FR-5	Agent feedback capture on AI suggestion quality
FR-6	Role-based access control via profiles and permission sets
FR-7	Reports and dashboards for SLA and agent performance monitoring
FR-8	Email alerts on case resolution

3.3 Non-Functional Requirements

NFR-1: Usability

The system should be easy to use for support agents, managers, and administrators without requiring technical training.

- Simple, intuitive Lightning interface with clear navigation.
- Agent-facing AI suggestions surfaced directly on the Case record.
- A new agent should be able to open a case, view the AI-suggested reply, and respond within minutes.

NFR-2: Security

The system must protect customer and case data from unauthorized access.

- Role-based access control through cloned profiles (Support Agent, Support Manager).
- Field-level security restricting edit access to sensitive fields such as SLA Due Date and AI Suggested Response.
- Dedicated permission sets that gate access to AI features and reporting tools.

NFR-3: Reliability

The system should work correctly at all times and store data accurately without loss.

- Record-triggered flows execute reliably on case creation and update.
- Validation rules prevent incomplete or inconsistent data from being saved.
- Salesforce's cloud infrastructure ensures automatic backups and data durability.

NFR-4: Performance

The system should respond quickly and handle case volumes efficiently.

- Case classification and SLA calculation complete instantly on save via record-triggered flows.
- Reports and dashboards render within a few seconds even with a growing case volume.

NFR-5: Availability

The system should be accessible whenever support agents need it.

- Hosted entirely on Salesforce's cloud platform with high uptime guarantees.
- Accessible from desktop and mobile Salesforce clients.

NFR-6: Scalability

The system should support growth in case volume and support team size without redesign.

- Custom objects and flows are built generically so new issue types, SLA policies, or agents can be added without structural changes.
- Salesforce's multi-tenant architecture scales automatically as data volume grows.

3.4 Data Flow

The system follows a simple, linear data flow: a customer or agent creates a Case, which is evaluated by validation rules and record-triggered flows for classification, priority, and SLA calculation. The Agentforce agent then reads the case and produces a suggested reply via a Prompt Template. Once resolved, an email alert notifies the customer, and the case data feeds into reports and dashboards for management visibility.

Case Creation → Validation Rules → Record-Triggered Flow (Classification, Priority, SLA) → Agentforce / Prompt Builder (Suggested Reply) → Agent Review & Resolution → Email Alert → Reports & Dashboards.

3.5 Technology Stack

Layer	Technology
Frontend	Salesforce Lightning Experience
Database	Salesforce Cloud (Standard & Custom Objects)
Automation	Record-Triggered Flows, Validation Rules, Email Alerts
AI / Intelligence	Agentforce, Prompt Builder, Prompt Templates
Security	Profiles, Permission Sets, Field-Level Security
Analytics	Reports & Dashboards

4. Project Design Phase

4.1 Problem–Solution Fit

Manual case handling is prone to inconsistent classification, missed SLAs, and slow, inconsistent responses. The proposed solution replaces this with automated classification and SLA tracking through record-triggered flows, and augments agent responses with AI-generated suggested replies via Agentforce. This ensures every case is handled consistently, on time, and with a higher baseline quality of communication.

4.2 Proposed Solution

Capability	Description
Case Management	Create, view, update, and track customer support cases end-to-end.
Automatic Classification	Record-triggered flow evaluates case details and assigns issue type and priority.
SLA Due Date Calculation	Flow automatically calculates the SLA due date based on priority and policy.
AI Suggested Reply	Agentforce Action backed by a Prompt Template drafts a professional, ready-to-review reply.
AI Feedback Capture	Agents log whether the AI suggestion was helpful, feeding a continuousimprovement loop.
Validation Rules	Enforce mandatory fields, such as SLA Due Date for High priority cases.
Email Alerts	Automated case-resolution email sent to the customer when status changes to Closed.
Role-Based Security	Profiles and permission sets control access for agents, managers, and administrators.
Reports & Dashboards	Real-time visibility into open cases by priority and agent performance.

4.3 Solution Architecture

The solution follows a layered architecture that keeps the presentation, automation, data, AI, and analytics concerns cleanly separated, which makes the system easier to maintain and extend:

- Presentation Layer — Salesforce Lightning App and Case UI
- Application Layer — Record-Triggered Flows and Validation Rules
- AI Layer — Agentforce Agent, Topic, Action, and Prompt Template
- Data Layer — Standard and Custom Objects (Case, Support Category, SLA Policy, AI Feedback)
- Security Layer — Profiles, Permission Sets, Field-Level Security
- Analytics Layer — Reports and Dashboards

This layered architecture ensures modularity, scalability, and maintainability — each layer can be modified independently without disrupting the others.

5. Project Planning & Scheduling

5.1 Execution Roadmap

The project was executed in eight structured phases to ensure smooth development, testing, and deployment:

Phase	Focus Area
Phase 1	Requirement Analysis & Planning — understand business needs, map stakeholders, finalize scope and data model.
Phase 2	Environment Setup — create the Salesforce Developer Org and enable Lightning Experience.
Phase 3	Data Model & Security — create objects, fields, relationships, profiles, and permission sets.
Phase 4	Automation & Classification — build record-triggered flows for case classification, priority, and SLA.
Phase 5	Agent Assistance & Feedback — configure Agentforce, Prompt Builder, and the AI feedback loop.
Phase 6	Reporting & Monitoring — build reports for open cases, SLA breaches, and agent performance.
Phase 7	Testing & Validation — verify automation, data integrity, and AI responses across scenarios.
Phase 8	Deployment & Documentation — activate configurations and finalize documentation for demonstration.

5.2 Product Backlog & Sprint Plan

The project followed an Agile-based approach, broken into sprints aligned with the milestones of the execution roadmap.

Sprint	Epic	User Story	Story Points	Priority
Sprint-1	Environment Setup	As an admin, I want to create a Salesforce Developer Org so that I can configure and deploy the CST application.	1	High
Sprint-1	Data Modeling	As an admin, I want to create Case-related standard and custom objects so support data can be stored systematically.	3	High

Sprint-2	Data Modeling	As a user, I want dedicated tabs and a Lightning App so I can navigate CST records easily.	3	High
Sprint-2	Data Modeling	As an admin, I want fields and relationships across Case, Support Category, SLA Policy, and AI Feedback so complete case data is captured.	5	High
Sprint	**Epic**	**User Story**	**Story Points**	**Priority**
Sprint-3	Automation	As a system, I want to validate case data so incomplete high-priority cases cannot be saved without an SLA date.	3	High
Sprint-3	Automation	As a system, I want a record-triggered flow to classify cases, assign priority, and calculate SLA due dates automatically.	5	High
Sprint-4	AI Integration	As an agent, I want an Agentforce assistant to suggest a reply for a case so I can respond faster.	5	High
Sprint-4	AI Integration	As an agent, I want to log feedback on AI suggestions so the system's effectiveness can be tracked.	3	Medium
Sprint-5	Automation	As a system, I want to send an automated email when a case is closed so customers are notified of resolution.	3	High
Sprint-5	Security	As an admin, I want profiles and permission sets so agents, managers, and admins have appropriate access.	5	High
Sprint-6	Reporting	As a manager, I want reports on open cases by priority and agent performance so I can monitor SLA compliance.	4	High

6. Project Development Phase

The development phase was executed across twelve milestones, moving from environment setup through data modeling, automation, AI integration, security, and reporting. Each milestone below is documented with the configuration steps performed and a screenshot verifying the completed implementation.

Milestone 1 – Salesforce Developer Account & Environment Setup

A Salesforce Developer Org was created to serve as the environment for building and testing the entire application, free of cost and fully featured for configuration, automation, and Agentforce.

- Signed up at developer.salesforce.com/signup with role, company, and country details.
- Verified the account via the activation email and set a secure password and security question.
- Landed on the Salesforce Setup Home page, confirming the org was ready for configuration.

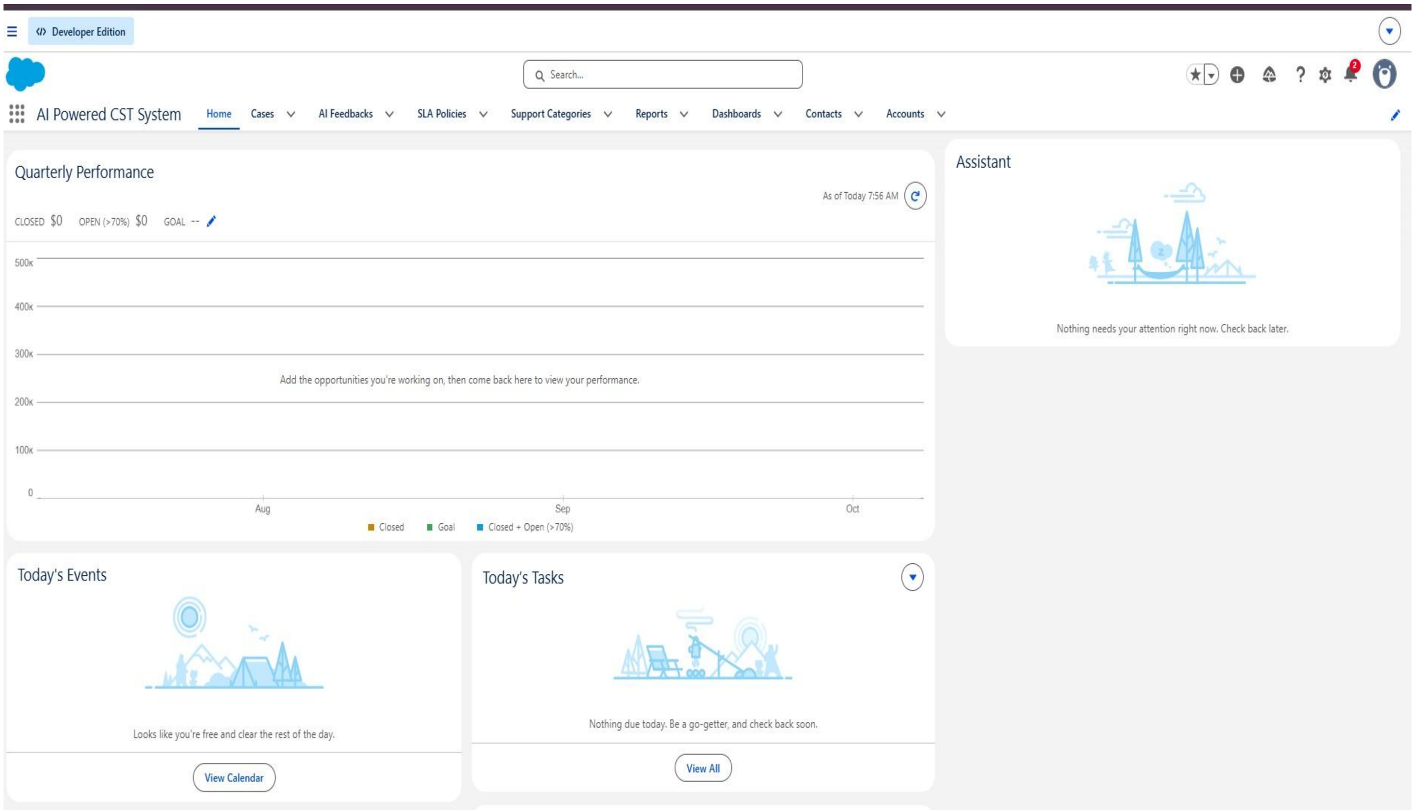


Figure 1: Salesforce Setup Home page after successful Developer Org activation.

Milestone 2 – The Lightning App

A dedicated Lightning App, "AI Powered CST System," was created to bring together all the objects, reports, dashboards, and the Agentforce assistant into a single, agent-friendly workspace.

- Configured app name, branding, and description in App Manager.
- Added Case, Support Category, SLA Policy, AI Feedback, Reports, and Dashboards as navigation items.
- Assigned the System Administrator profile for app access and saved the configuration.

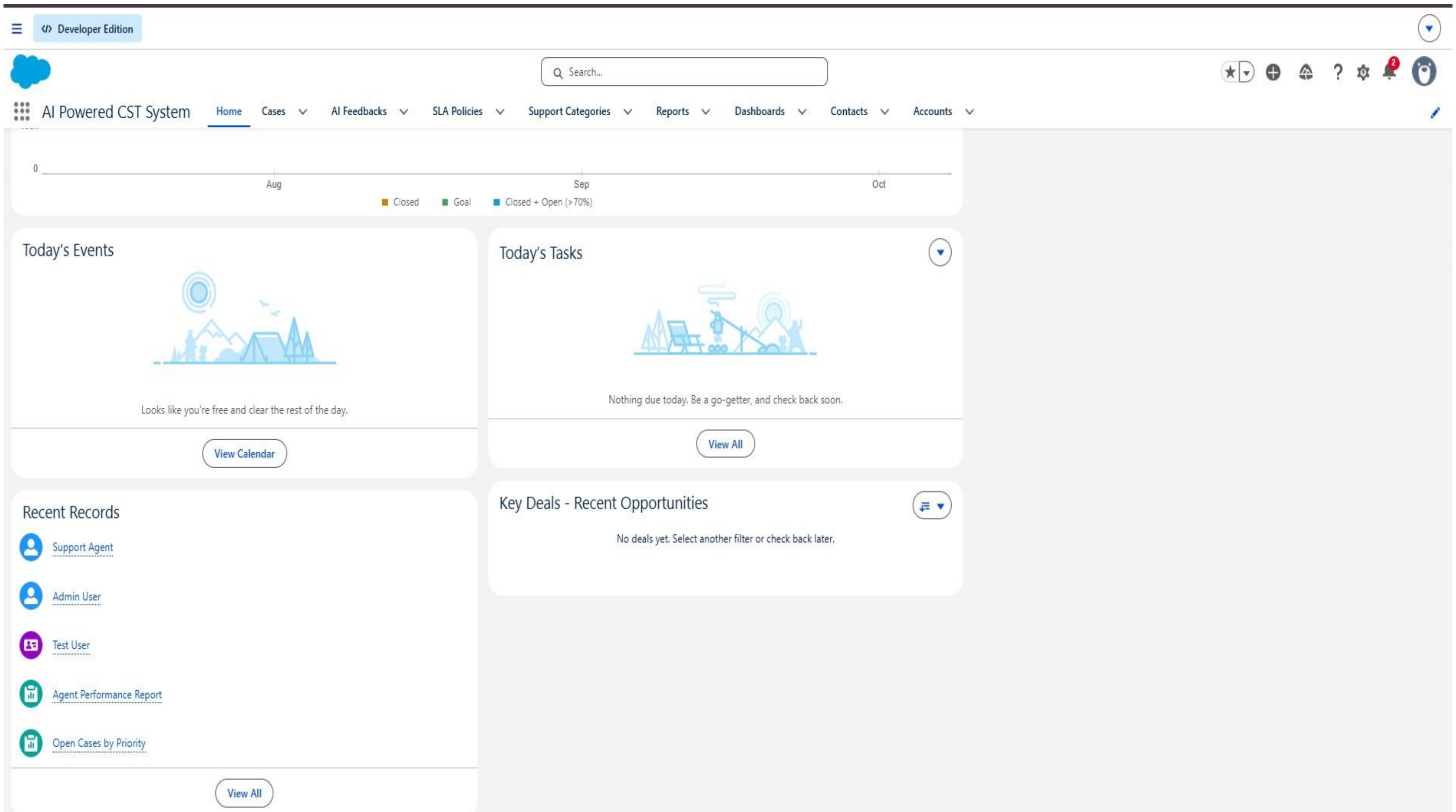


Figure 2: The AI-Powered CST System Lightning App with navigation tabs and Agentforce assistant icon.

Milestone 3 – Object Model

The data model combines Salesforce's standard objects with three purpose-built custom objects to capture everything the AI-powered support process needs.

Standard Objects Used

Object	Key Fields
Account	Account Name, Phone, Industry, Type
Contact	First Name, Last Name, Email, Phone
Case	Case Number, Subject, Description, Status, Priority, Issue Type, SLA Due Date, AI Suggested Response, Resolution Time

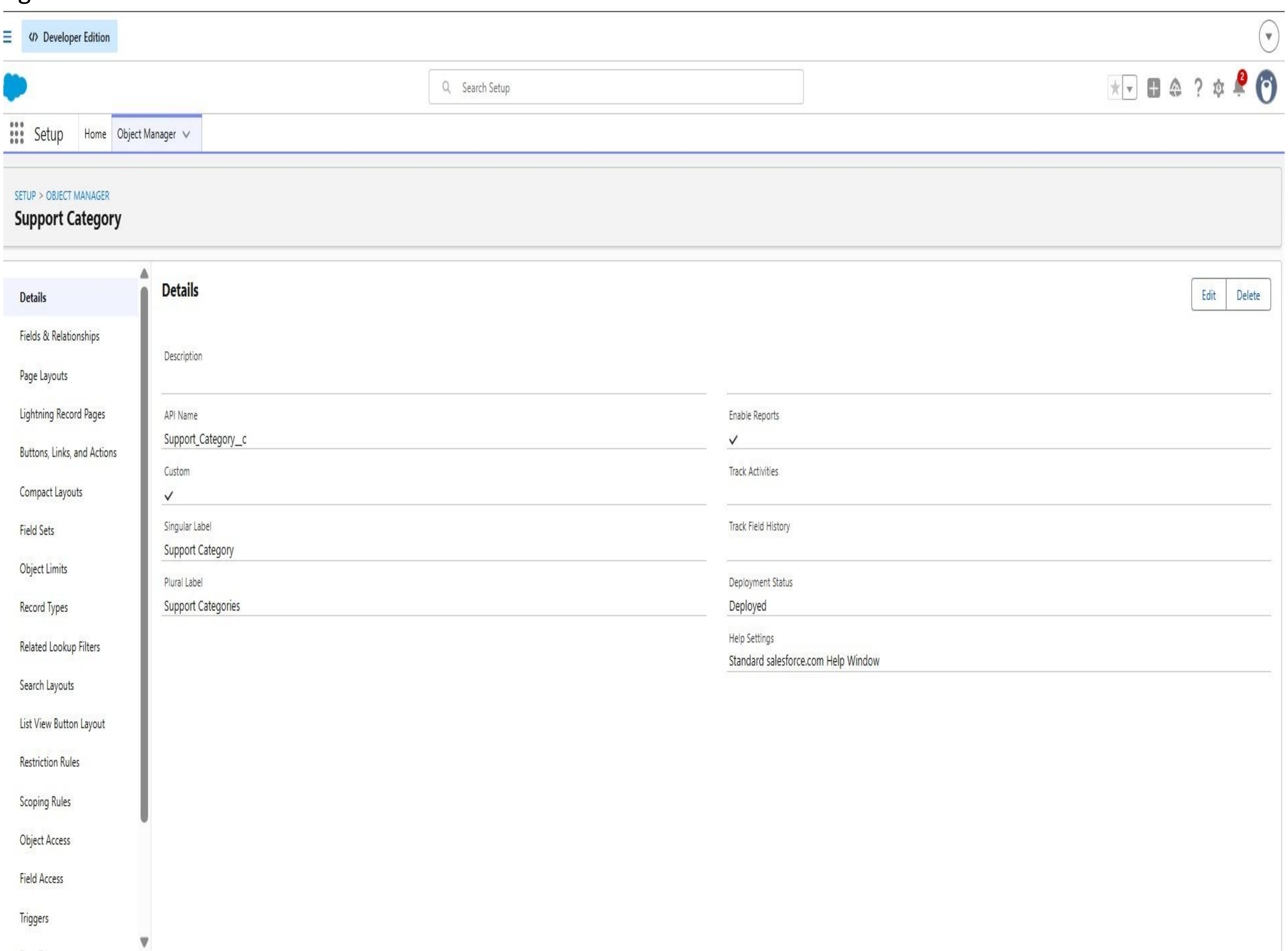
Custom Objects Created

Object	Key Fields
Support_Category__c	Category Name, Description, Default Priority, Active
SLA_Policy__c	Policy Name, Priority Level, Resolution Time (Hours), Escalation Required
AI_Feedback__c	AI Feedback Number (Auto Number), AI Suggestion Type, Feedback Status, Agent Comments

Object Relationships

Relationship	Type
Account → Contact	Lookup
Account → Case	Lookup
Contact → Case	Lookup
Support Category → Case	Lookup
SLA Policy → Case	Referenced via Flow
AI Feedback → Case	Lookup

Custom tabs were created for each custom object (Support Category, SLA Policy, and AI Feedback) so that support agents and administrators coul



d access and manage these records directly from the Lightning App navigation bar.

Figure 3: Support Category custom object records, used to drive default case classification.

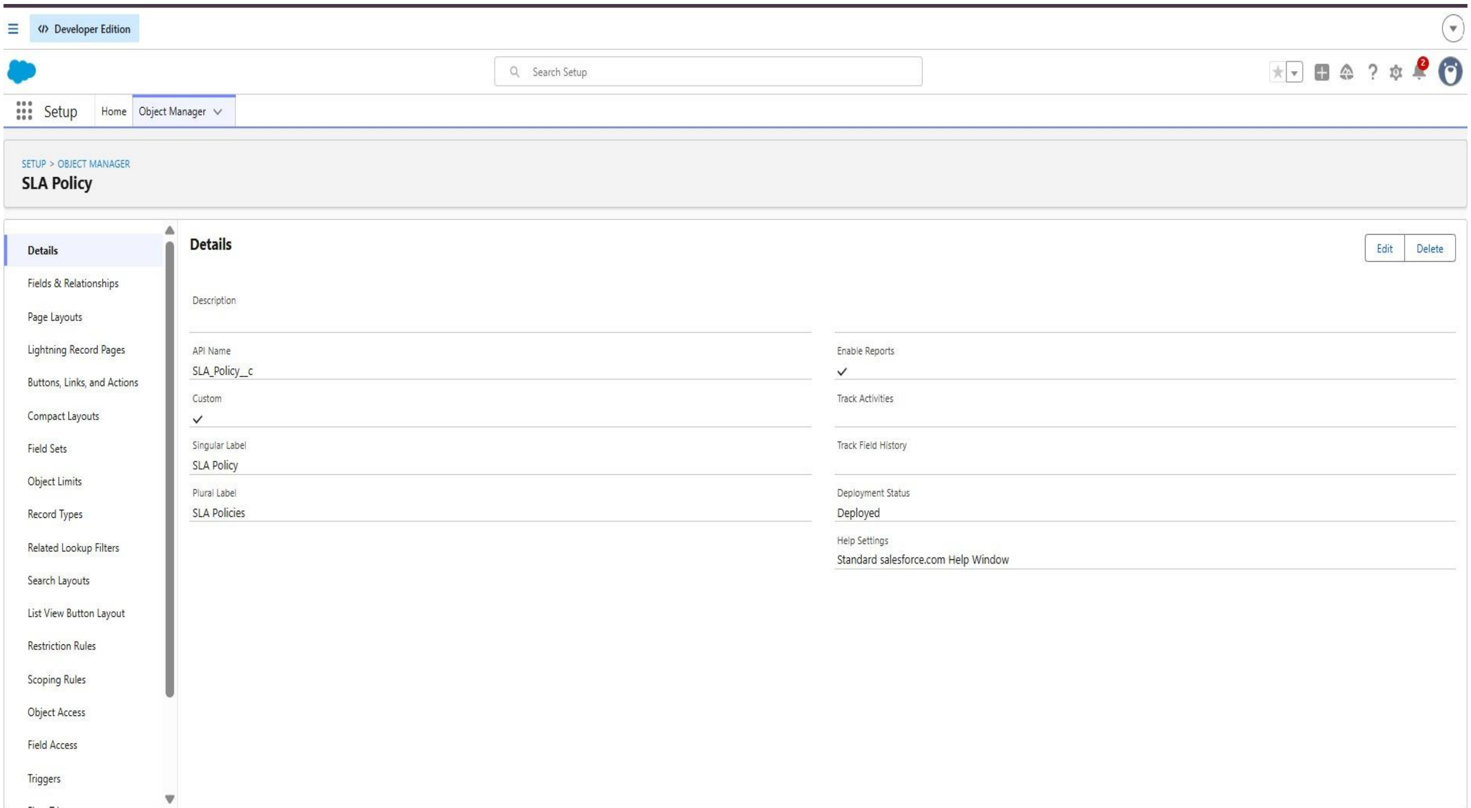


Figure 4: SLA Policy custom object, defining resolution time and escalation rules by priority level.

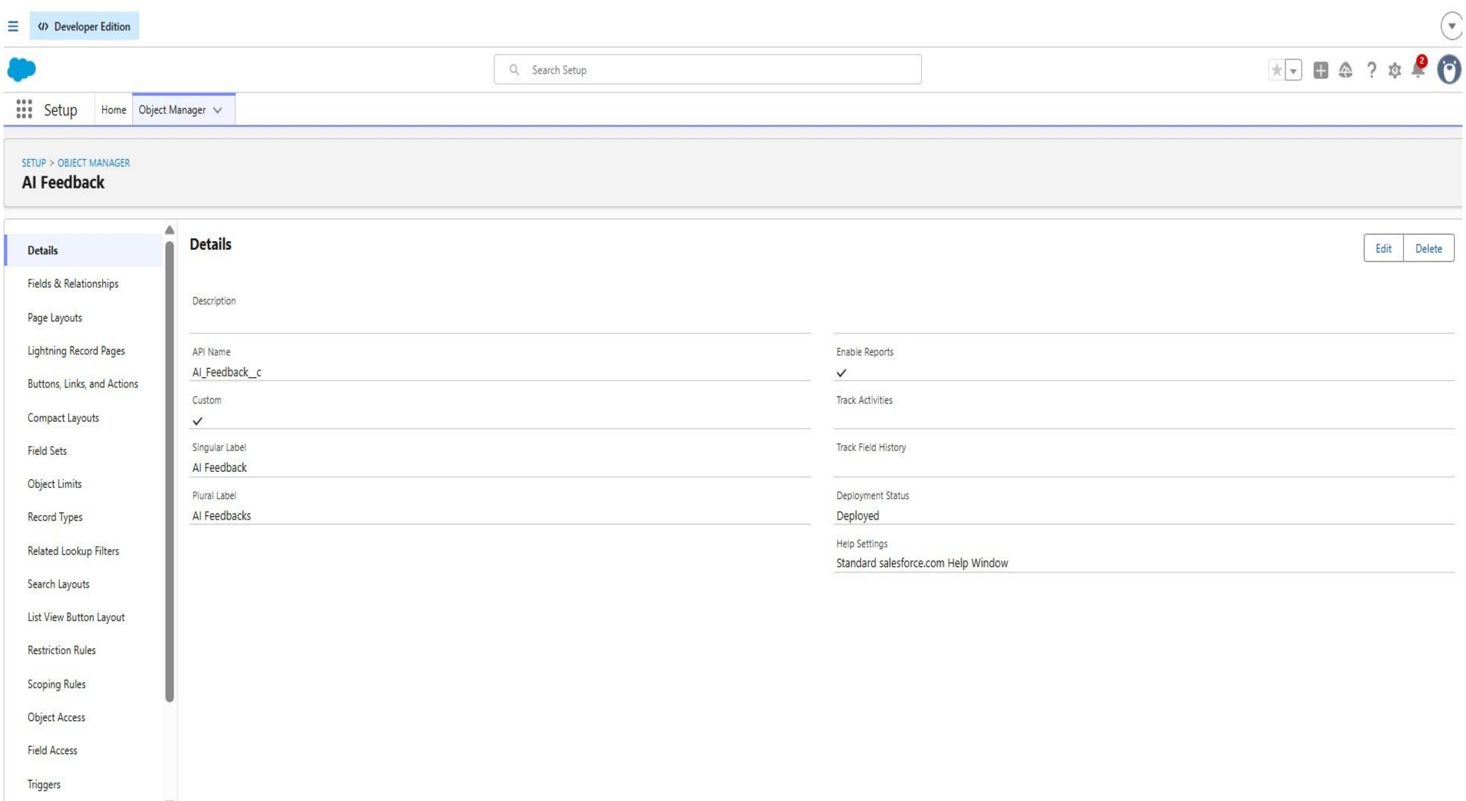


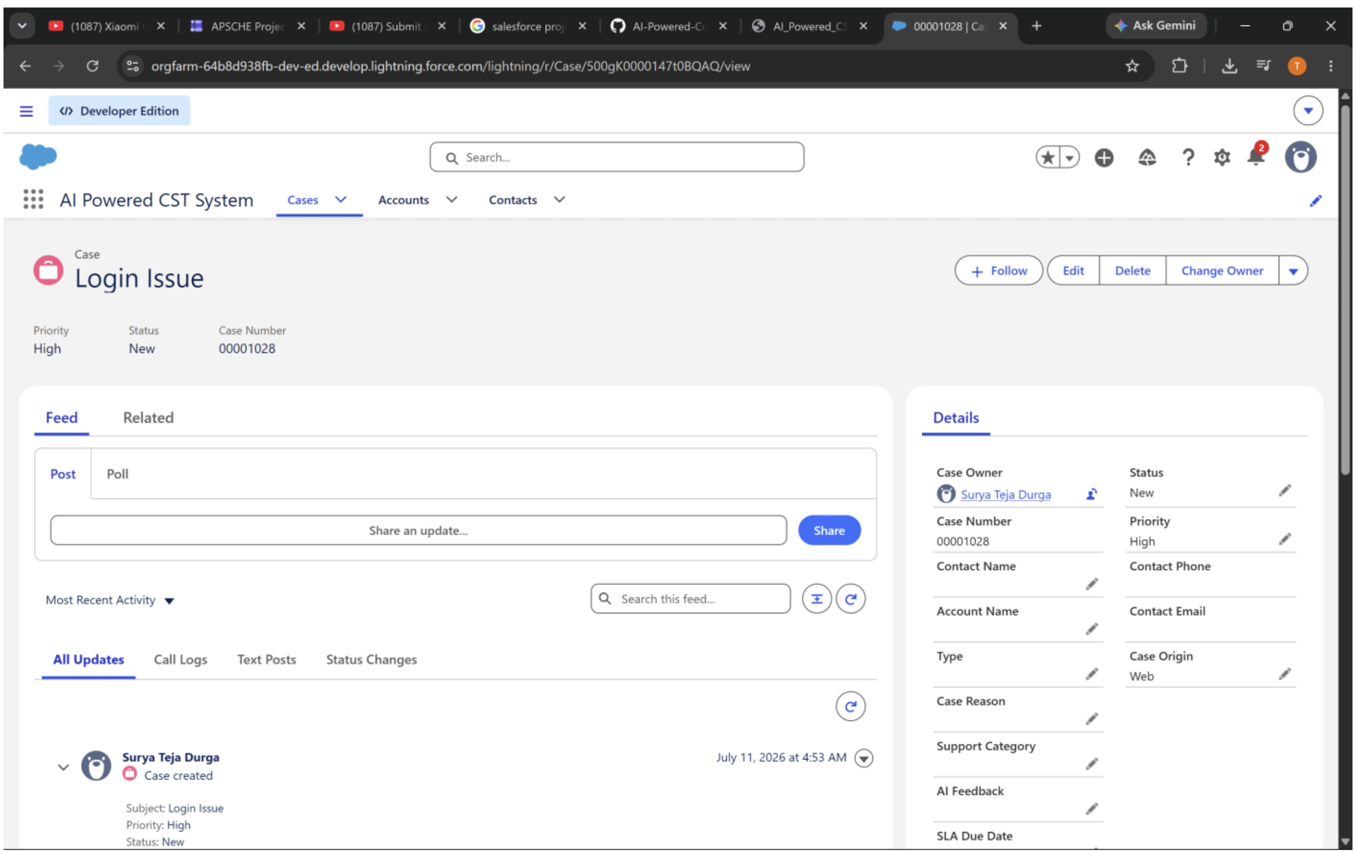
Figure 5: AI Feedback custom object capturing agent ratings of AI-generated suggestions.

Milestone 4 – Fields & Relationships (AI Feedback)

The AI_Feedback__c object was configured with the following fields to capture structured feedback on AI performance:

Field Label	API Name	Data Type	Purpose
AI Feedback Number	Name	Auto Number	Unique feedback record identifier
Case	Case__c	Lookup (Case)	Related support case
Feedback Status	Feedback_Status__c	Picklist (Helpful, Not Helpful)	Agent rating of AI output
AI Suggestion Type	AI_Suggestion_Type__c	Picklist	Case Classification / Priority Prediction / Suggested Reply
Agent Comments	Agent_Comments__c	Long Text Area	Agent remarks on AI quality
Created By	CreatedById	Lookup (User)	Agent who provided feedback

Milestone 5 – Case Record & Data Entry



A test Case record was created to walk through the full lifecycle: entry of subject and description, automatic classification, priority assignment, SLA due date calculation, and the AI-suggested response field, all visible on a single Lightning record page.

Figure 6: A Case record showing captured issue details alongside the auto-classified priority, SLA due date, and AI Suggested Response field.

Milestone 6 Validation Rules

A validation rule, SLA_Due_Date_Required_For_High_Priority, was implemented on the Case object to guarantee that no High-priority case can be saved without an SLA due date — preventing accidental SLA breaches caused by missing data.

Validation Formula:

AND(ISPICKVAL(Priority, "High"), ISBLANK(SLA_Due_Date__c))

- Error Message: "SLA Due Date must be populated for High priority cases."
- Error Location: SLA Due Date field.
- Negative test: creating a High priority case with a blank SLA Due Date correctly blocked the save.
- Positive test: entering a valid SLA Due Date allowed the case to save successfully.

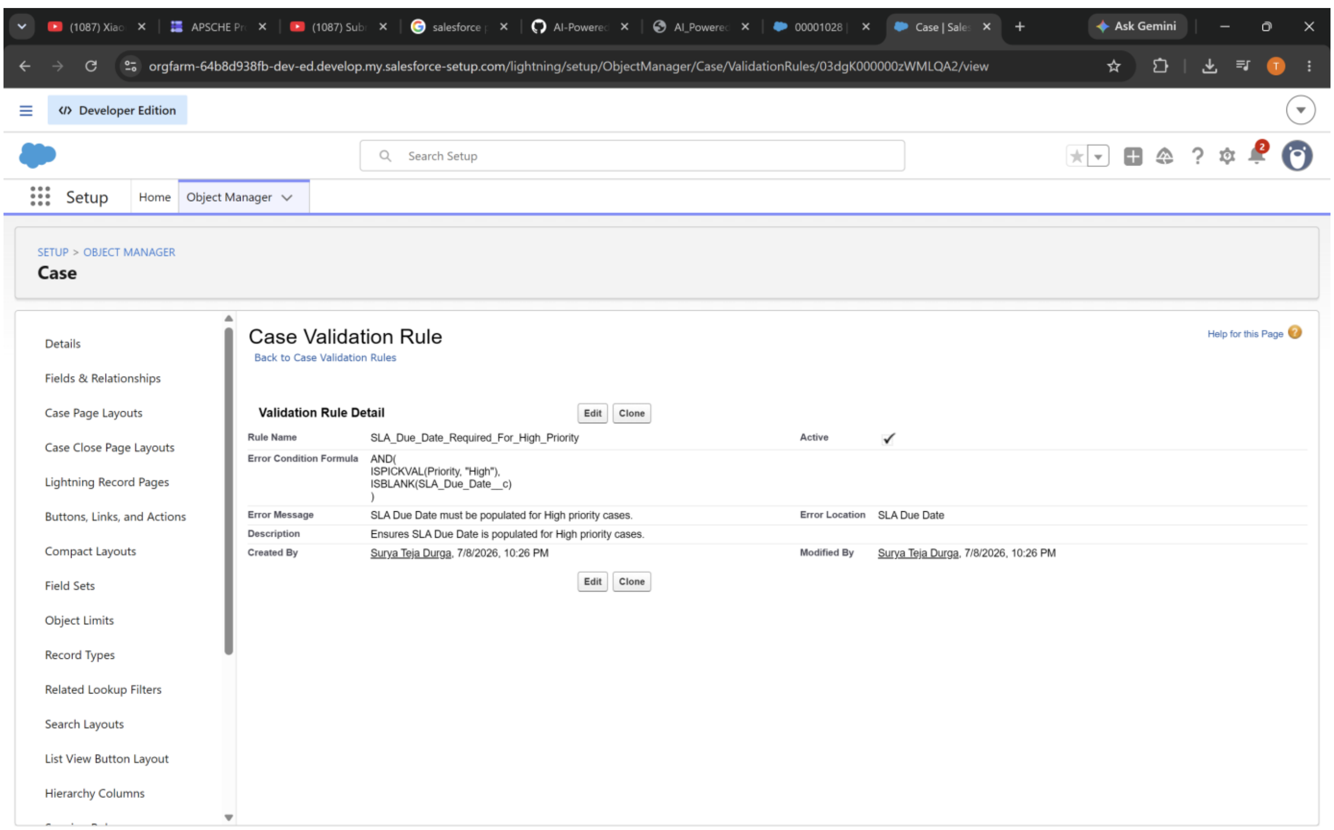


Figure 7: The SLA Due Date validation rule configured on the Case object in Object Manager.

Milestone 7 – Automation: Record-Triggered Flow

A record-triggered flow was built to automatically classify each incoming case, assign the correct priority, and calculate the SLA due date the moment a case is created — removing this repetitive work from agents entirely.

- Object: Case; Trigger: When a record is created.
- Decision logic reads the case Subject/Description and Support Category to set Issue Type and Priority.
- SLA Due Date is calculated by adding the SLA Policy's Resolution Time (Hours) to the case's created date/time.
- The flow was activated and tested by creating cases across Low, Medium, and High priority scenarios, confirming correct classification and SLA calculation in each case.

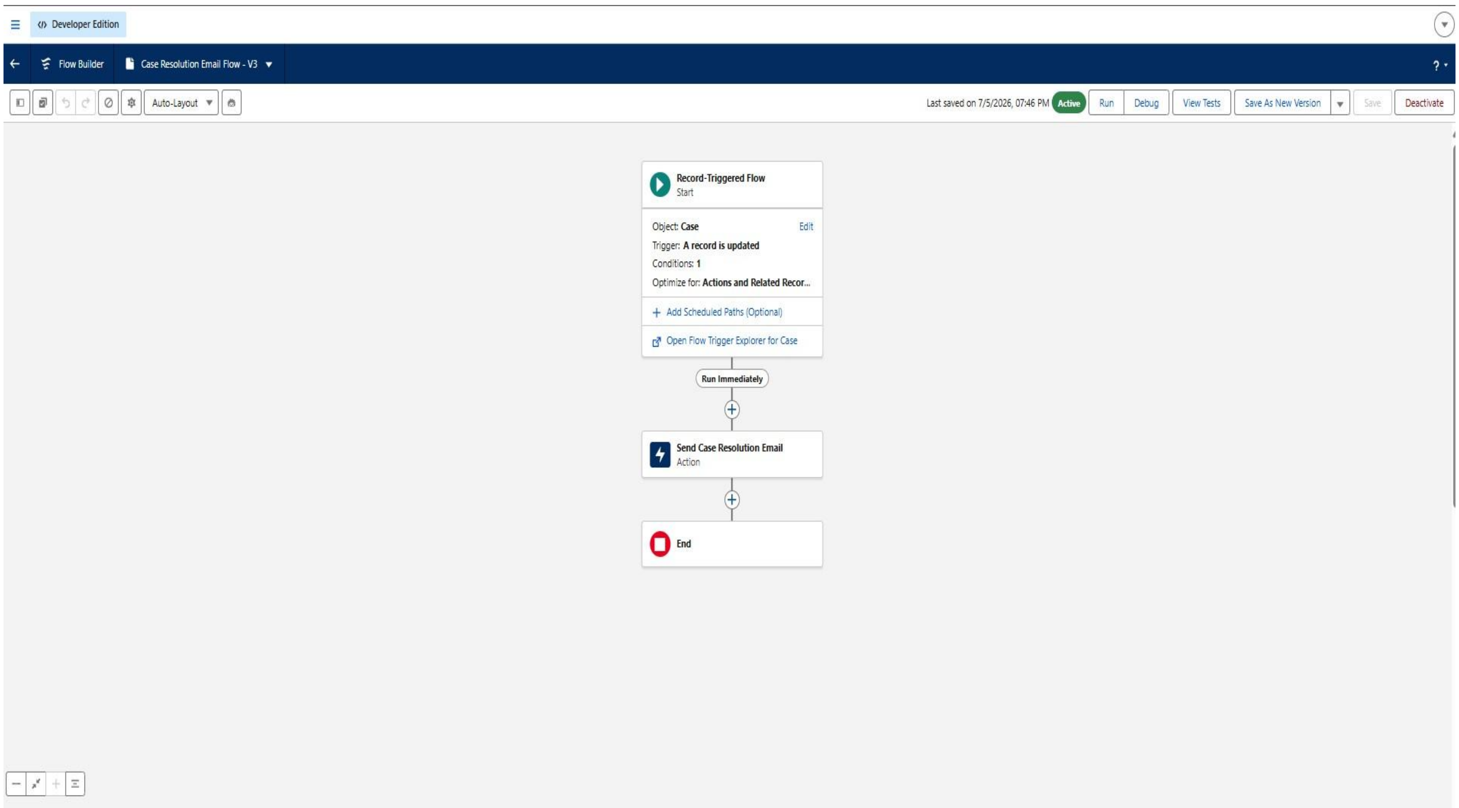


Figure 8: The record-triggered Flow in Flow Builder, showing the classification, priority, and SLA calculation logic.

Milestone 8 – Email Templates & Alerts

To keep customers informed once their issue is resolved, a Case Resolution Email was implemented using a classic Email Template and an Email Alert, triggered by a record-triggered flow whenever a case's Status changes to Closed.

Email Template: Case Resolution Email

- Subject: “Your Case Has Been Resolved – {!Case.CaseNumber}”
- Body includes the customer's name, case number, and current status, with a closing thank-you message.

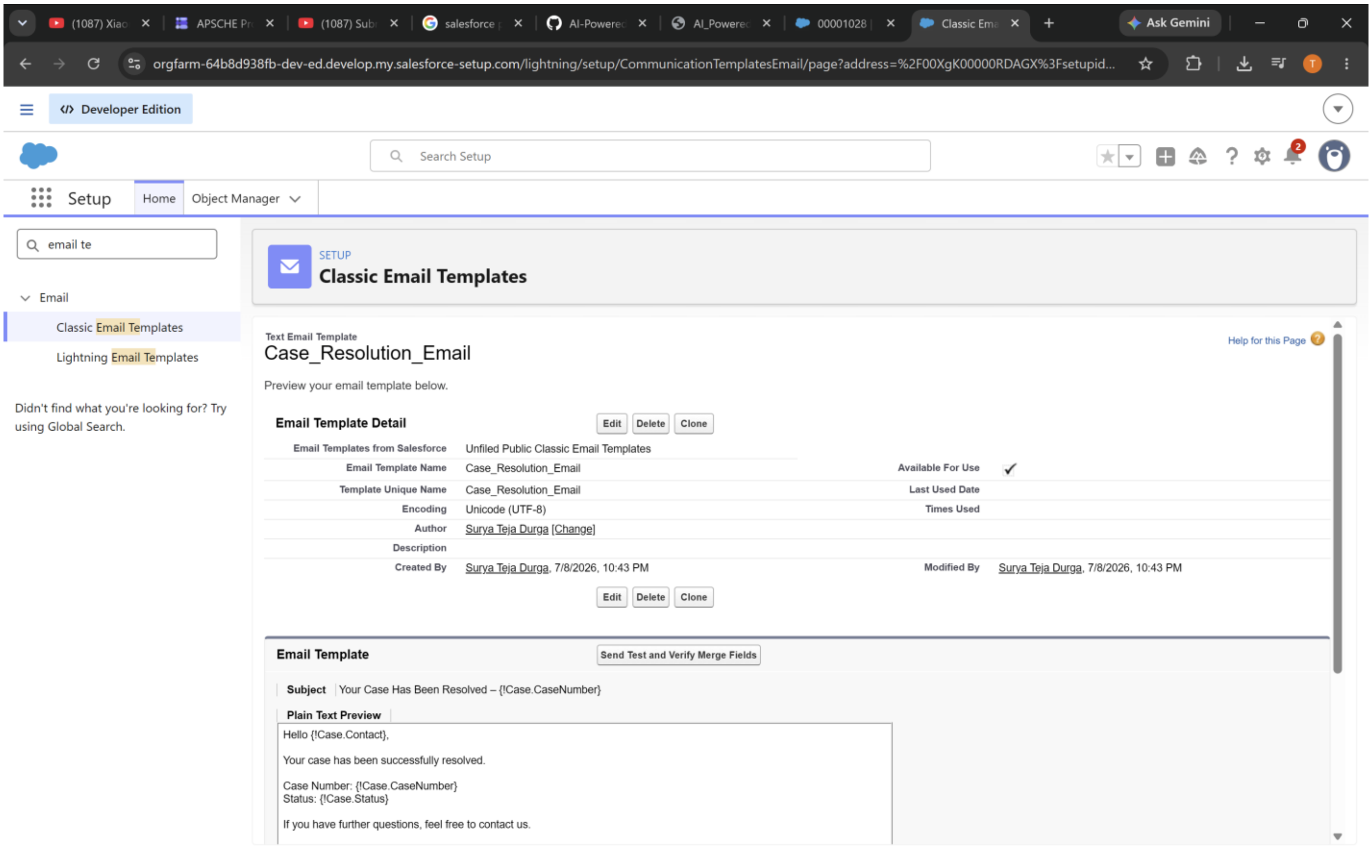


Figure 9: The Case Resolution Email classic email template configured with merge fields.

Email Alert Configuration

- Object: Case; Email Template: Case_Resolution_Email; Recipients: Contact Email.
- Triggered by a record-triggered flow with the condition Status Equals Closed, using "Only when the record is updated to meet the condition requirements" to avoid duplicate emails.

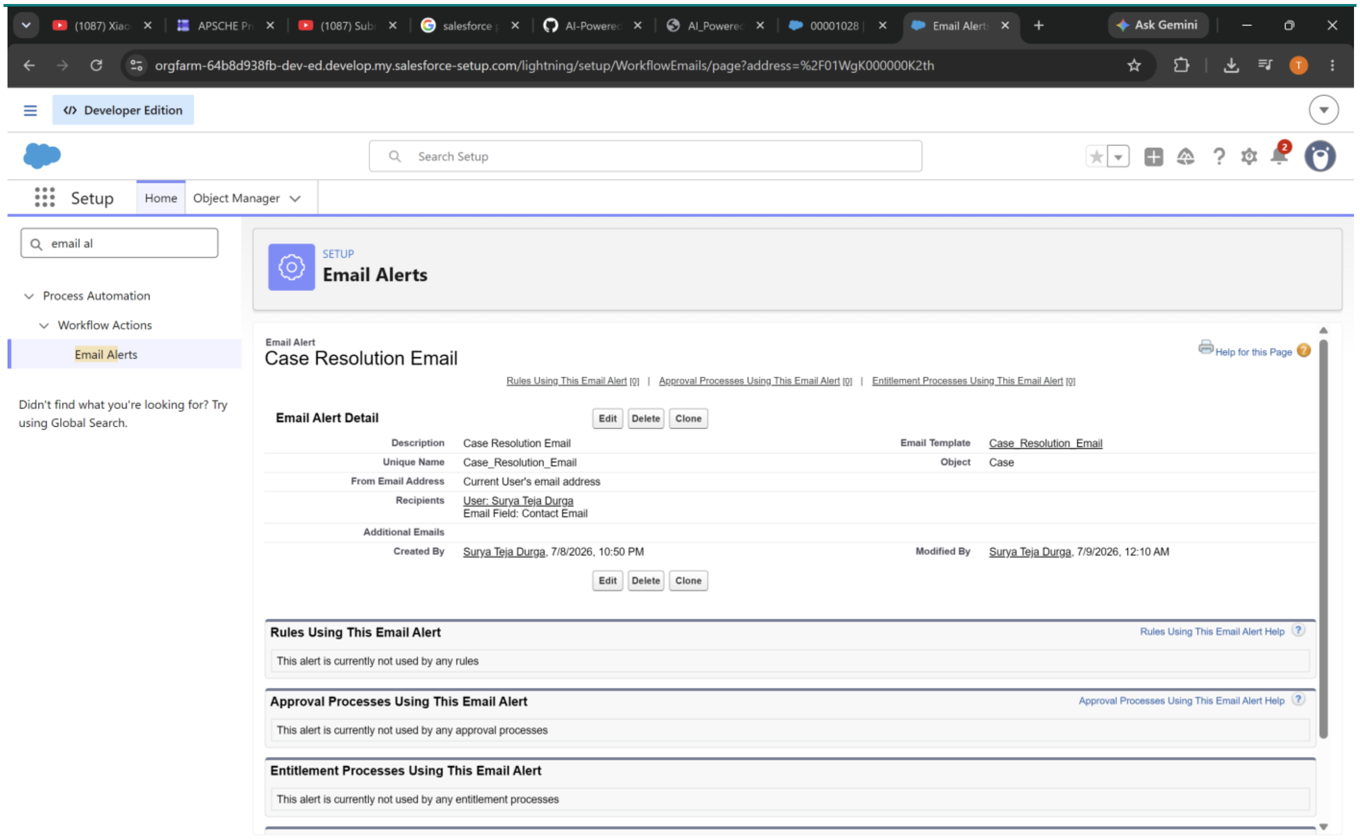


Figure 10: The Case Resolution Email Alert configured with its linked template and recipient.

Testing: An existing case with a non-Closed status was updated to Closed. The flow fired correctly and the Case Resolution Email was delivered to the associated Contact's email address, confirming end-to-end automation.

7. Agentforce AI Implementation

The AI capability at the heart of this project is delivered through Agentforce, Salesforce's native AI agent platform, combined with Prompt Builder for structured, field-aware prompt generation. Two AI features were implemented: automatic case classification support and AI-suggested replies for agents.

7.1 How Agentforce Works in This Project

- A Case is created with a Subject, Description, Issue Type, and Priority.
- The Agentforce Agent, when triggered on the Case record, reads the case description via a Prompt Template.
- The Prompt Template asks the model to act as a customer support agent and draft a polite, professional, and helpful reply based on the Issue Type, Priority, and Description.
- The generated reply is displayed to the agent, either directly on the Case record (via the AI Suggested Response field) or through the conversational Agentforce Agent panel.
- The agent reviews, edits if needed, and sends the final reply to the customer.

7.2 Enabling Agentforce and Prompt Builder

- Setup → Einstein Setup → toggled Turn on Einstein to enable AI features org-wide.
- Setup → Prompt Builder → enabled Prompt Templates.

7.3 Prompt Template: AI Suggested Response

A Field Generation prompt template was created against the Case object, targeting a new long-text field, AI_Suggested_Response__c, added specifically to store the AI's draft reply.

Prompt used:

"You are a customer support agent. Based on the following case details, generate a polite, professional, and helpful reply. Issue Type: {!Case.Issue_Type__c}. Priority: {!Case.Priority}. Customer Description: {!Case.Description}. Provide a clear response that helps the customer resolve the issue."

The template was saved, activated, and assigned as the org default, then verified directly on the Case record by triggering the field's AI generation icon and confirming a coherent, context-aware reply was produced.

7.4 Agentforce Agent Configuration

The default Agentforce Agent was opened in Agent Builder and extended with a custom Topic and Action tailored to this use case:

Setting	Value
Topic Name	CST Topic
Classification Description	This topic is mainly about giving suggested responses for customer cases.
Scope	Providing solution/suggested responses for customers based on case details.
Setting	**Value**
Instructions	Run this topic when the user asks about a case's suggested reply; ask for the Case Id; then return the AI Suggested Response generated by the prompt template.

Agent Action

- Reference Action Type: Prompt Template; Reference Action: AI Suggested Response.
- Agent Action Label: AI Suggested Response.
- Instructions: first collect the Case Id from the user, then behave as a customer support agent and return a clear, helpful suggested reply.
- "Show in conversation" enabled so the generated reply displays directly in the chat with the agent.

The agent was activated and tested with prompts such as "Can you give a case solution response?", which triggers the agent to request a Case Id, then returns the AI-generated suggested reply for that specific case — confirmed working end-to-end inside the AI-Powered CST System app.

7.5 Outcome

The combination of Prompt Builder and a custom Agentforce Topic/Action gives support agents a conversational, on-demand way to retrieve an AI-drafted reply for any case simply by supplying the Case Id — without leaving the Salesforce interface. Agent feedback on these suggestions is captured through the AI_Feedback__c object, closing the loop between AI output and measured usefulness.

8. Reporting & Monitoring

Two reports were built to give support managers real-time visibility into case load and agent performance, both accessible from the AI-Powered CST System app.

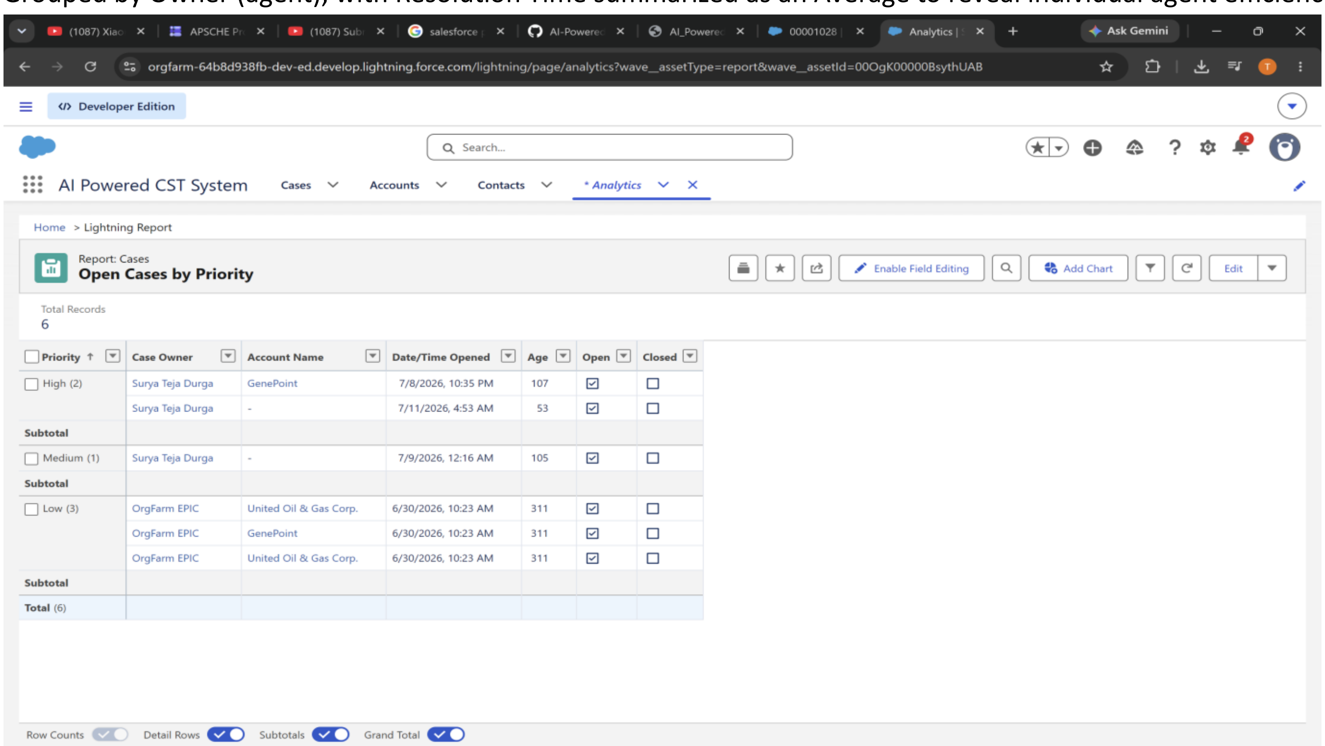
8.1 Open Cases by Priority

- Report Type: Cases; Filter: Status Not Equal To Closed.
- Columns: Case Number, Subject, Priority, Status, Case Owner.
- Grouped by Priority to give an immediate view of how many open cases fall into each priority tier.

8.2 Agent Performance Report

- Report Type: Cases; Filter: Status Equals Closed.
- Columns: Case Number, Owner, Priority, Created Date, Closed Date, Resolution Time (Hours).

Grouped by Owner (agent), with Resolution Time summarized as an Average to reveal individual agent efficiency.



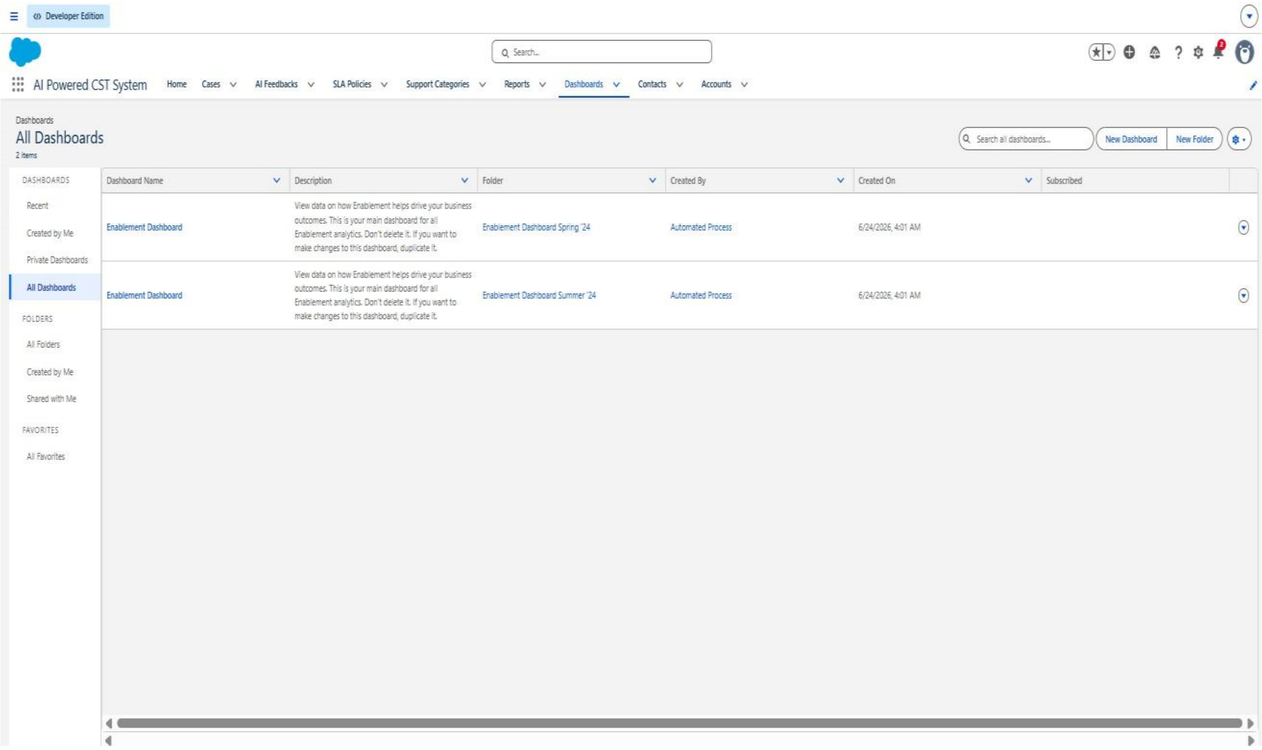


Figure 11: Open Cases by Priority and Agent Performance reports listed within the CST Reports folder.

8.3 Dashboard

A dashboard combining both reports was created to give managers a single-glance view of open case load by priority alongside agent resolution performance, refreshed in real time as case data changes.

Figure 12: Dashboard visualizing open case distribution by priority and agent performance metrics.

9. Security & Access Control

Role-based access control was implemented using cloned profiles for the two primary user roles — Support Agent and Support Manager — alongside dedicated permission sets that grant additional, feature-specific access. The System Administrator profile was used as-is for full administrative access.

9.1 Support Agent Profile

Cloned from Standard User. Configured with:

- Case: Read, Create, and Edit enabled; Delete disabled.
- AI_Feedback__c: Create and Read enabled; Edit and Delete disabled.
- Account and Contact: Read-only access.
- Field-level security: Status and Priority editable; SLA Due Date and AI Suggested Response read-only.

- App access: AI Powered CST System enabled.

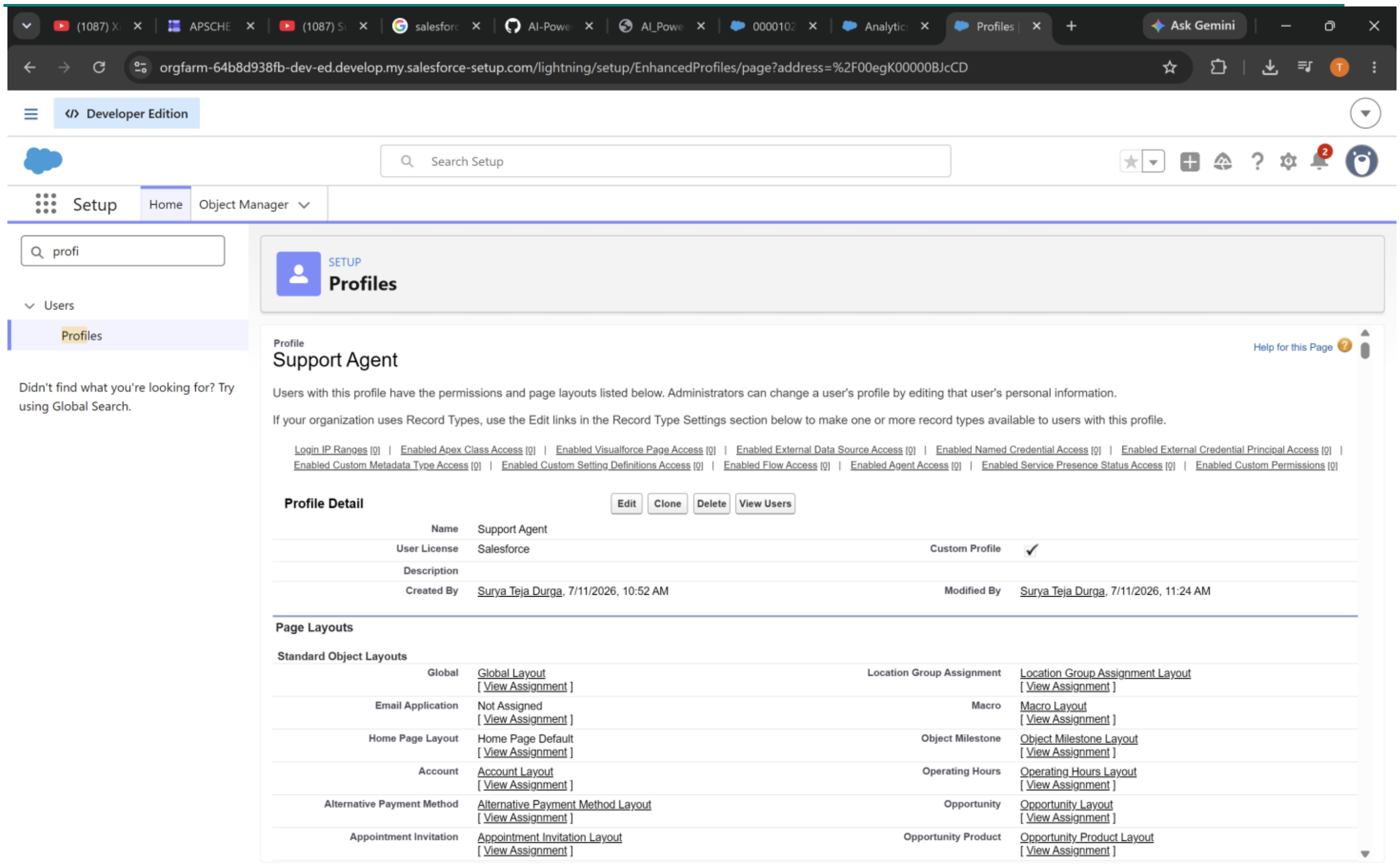


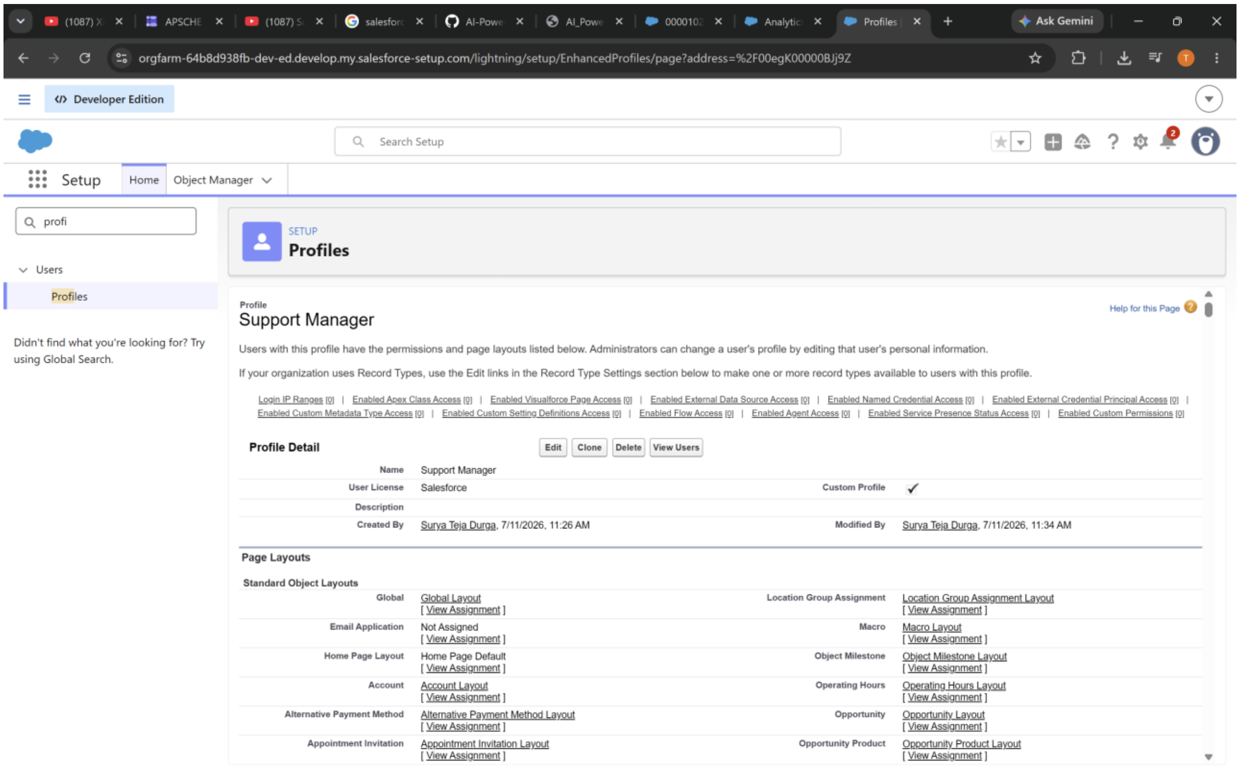
Figure 13: Support Agent profile configuration showing object and field-level permissions.

9.2 Support Manager Profile

Cloned from Standard User. Configured with:

- Case: Create, Read, Edit, and Delete all enabled.
- Reports & Dashboards: Create and Subscribe access enabled.
- Field-level security: Priority, Owner, and SLA Due Date editable; AI Suggested Response read-only.
- App access: AI Powered CST System enabled.

Figure 14: Support Manager profile configuration with elevated Case and reporting permissions.



9.3 Permission Sets

Two permission sets extend the base profiles with feature-specific access, following the recommended Salesforce pattern of using profiles for baseline access and permission sets for additional capabilities.

Permission Set	Grants	Assigned To
AI Feature Access	Edit AI-related Case fields; Create/Read on AI_Feedback__c; Access Einstein Features; Use Agent Assist	Support Agents
Report Access	Create and Customize Reports; Subscribe to Reports	Support Managers

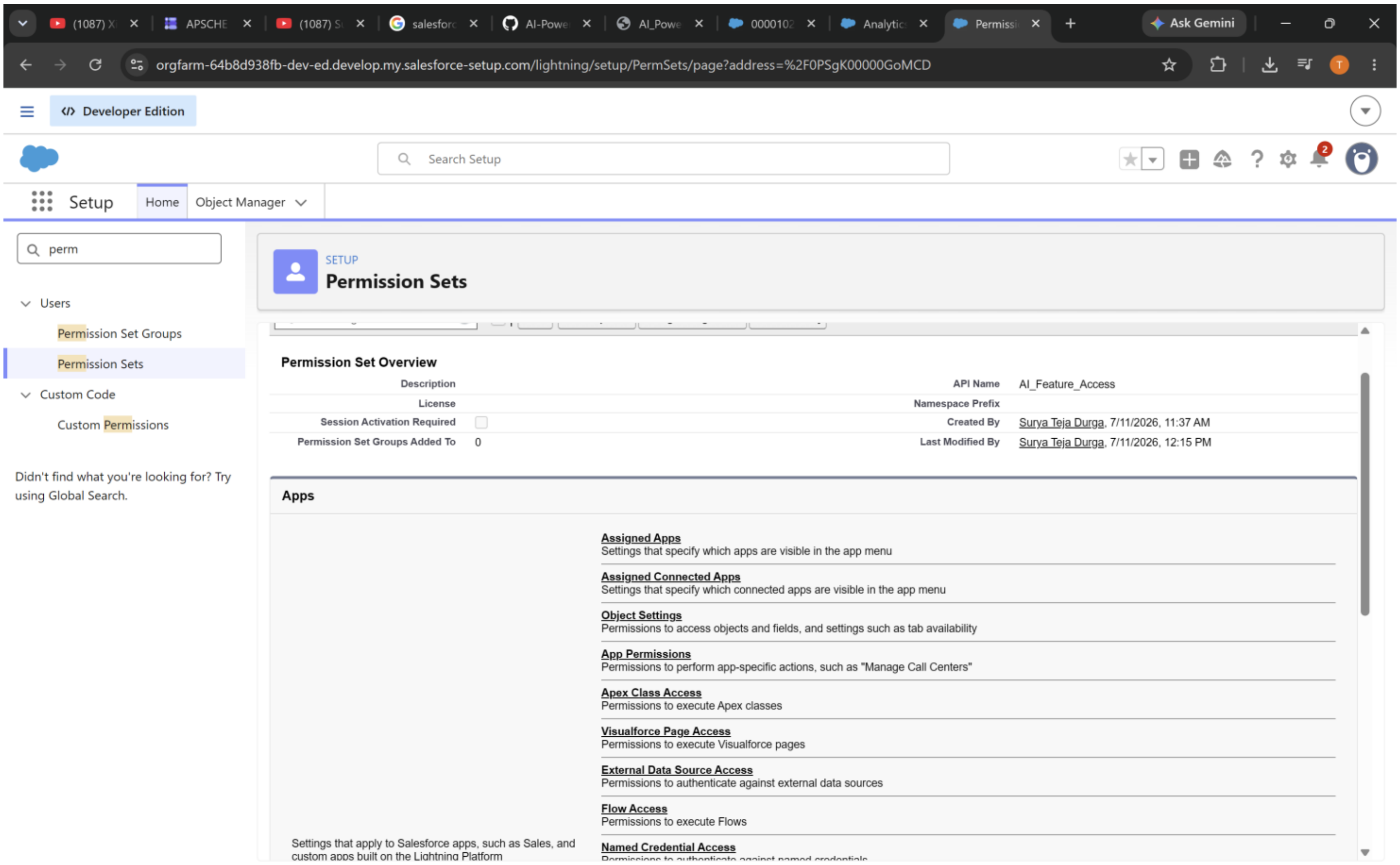


Figure 15: The AI Feature Access and Report Access permission sets configured in Setup.

9.4 Role – Profile – Permission Set Summary

Role	Profile	Permission Set
Support Agent	Support Agent	AI Feature Access
Support Manager	Support Manager	Report Access
Admin	System Administrator	All (native)

9.5 Users

Three users were created to represent the distinct roles in the system, each assigned the appropriate profile and permission set:

User	Profile	Permission Set Assigned
Admin User	System Administrator	—
Support Agent	Support Agent	AI Feature Access
Support Manager	Support Manager	Report Access

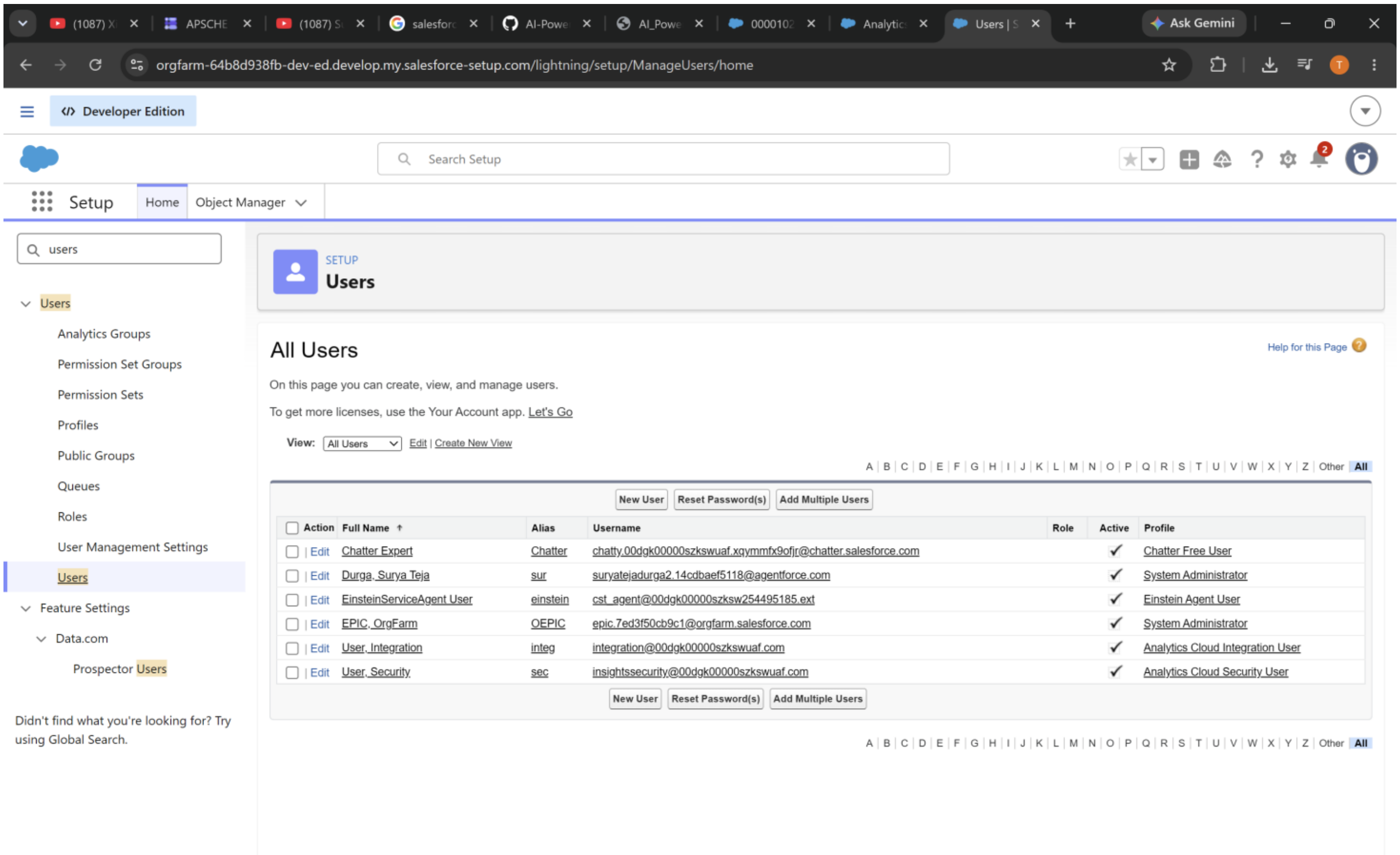


Figure 16: All three users listed in Setup → Users, with profiles assigned.

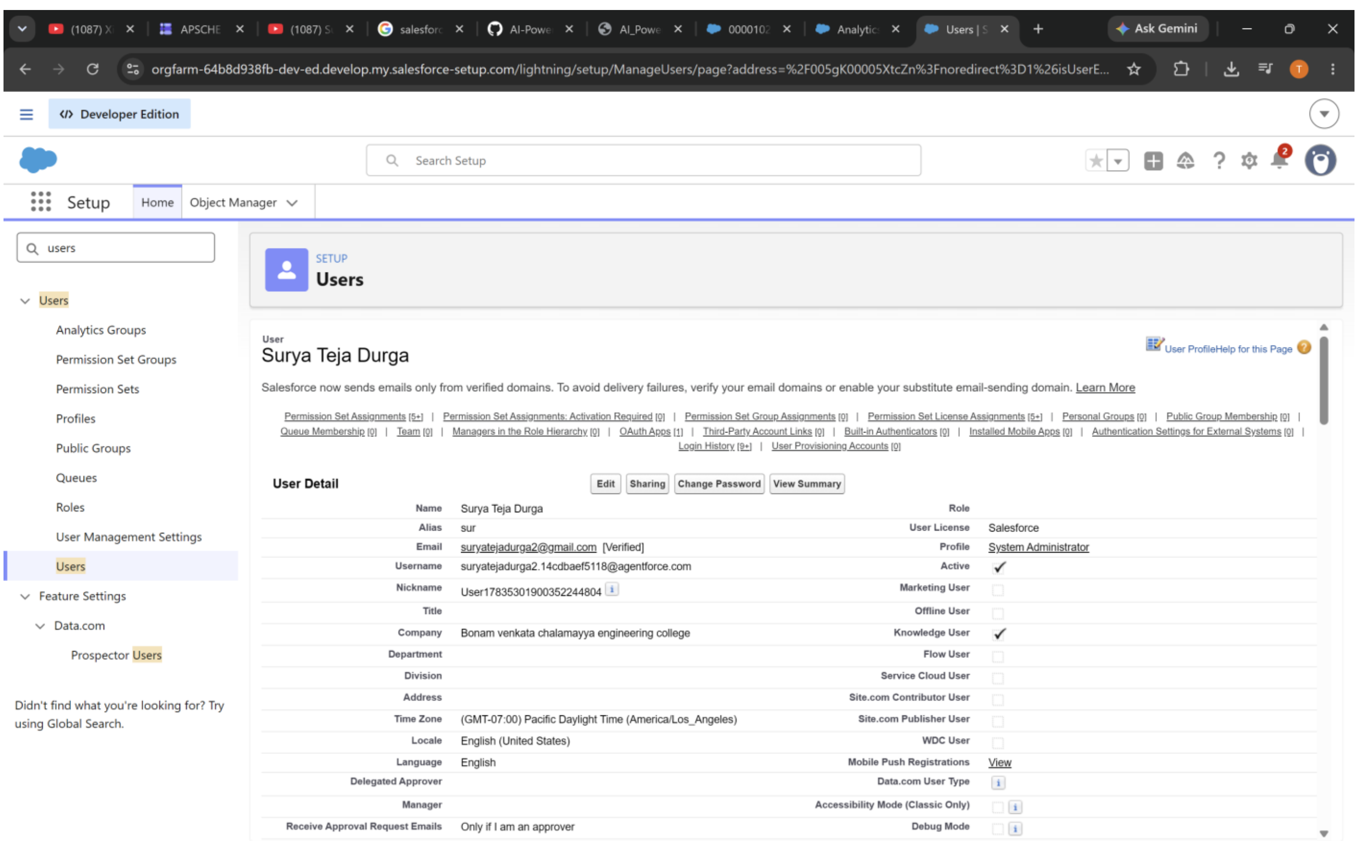


Figure 17: Admin User record configured with the System Administrator profile.

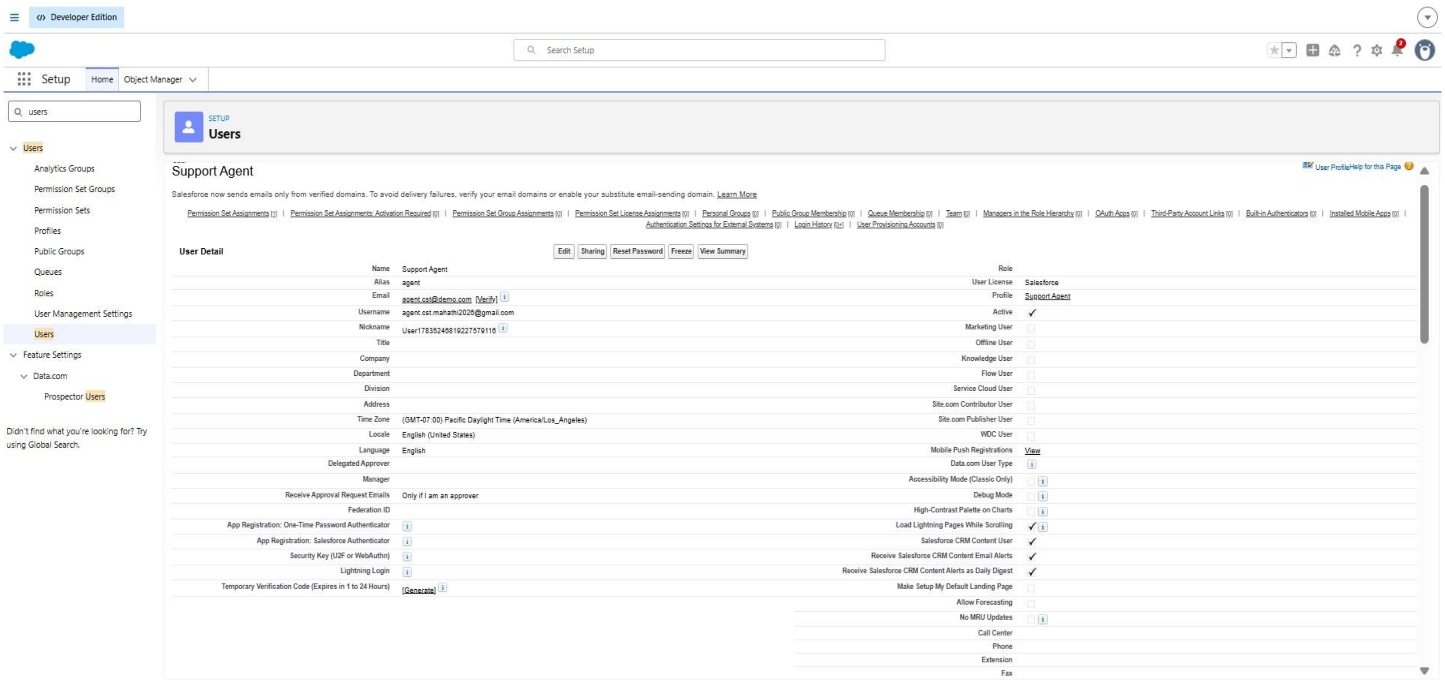


Figure 18: Support Agent user record configured with the Support Agent profile.

Final verification confirmed that each user has exactly one profile, is marked Active, has a unique username, and that the assigned profile correctly matches the intended role — satisfying the security design of the system.

10. Functional & Performance Testing

Testing was performed throughout the development phase to verify that each automation, validation rule, and AI feature behaved as designed. The following scenarios were executed and verified directly in the Salesforce Developer Org:

- Case creation across Low, Medium, and High priority scenarios, confirming correct automatic classification and SLA due date calculation.
- Validation rule testing: attempting to save a High priority case with a blank SLA Due Date was correctly blocked; supplying a date allowed the save to succeed.
- Agentforce testing: triggering the AI Suggested Response action with a valid Case Id returned a coherent, context-appropriate reply drafted from the case's Issue Type, Priority, and Description.
- AI Feedback testing: logging Helpful / Not Helpful feedback against a case correctly created an AI_Feedback__c record linked to that case.
- Email alert testing: updating a case's Status to Closed triggered the Case Resolution Email flow, and the email was delivered to the Contact's email address.

- Reports and dashboard testing: confirmed that Open Cases by Priority and Agent Performance reports reflected live case data accurately, and that the dashboard refreshed correctly.
- Security testing: logged in as each of the three test users to confirm object, field, and app-level access matched the intended profile and permission set configuration.

Performance testing confirmed that record-triggered flows executed instantly on save, and that reports and dashboards rendered within a few seconds even as case volume increased, with no noticeable delay for end users.

11. Maintenance, Monitoring & Troubleshooting

The Salesforce AI-Powered Customer Support Ticketing System requires ongoing maintenance and monitoring to ensure smooth operation, data accuracy, and reliable automation. Regular maintenance activities include reviewing and updating custom objects, fields, picklist values, and flows to align with changing business requirements. Profiles and permission sets are periodically audited to ensure proper access control and compliance with security best practices.

System monitoring focuses on tracking case volumes, SLA compliance, and automation performance using reports and dashboards. Flow executions are reviewed to confirm that case classification, priority assignment, and SLA calculations are functioning as expected. AI feedback data is monitored to evaluate the effectiveness of systemgenerated suggestions and identify areas for improvement.

Troubleshooting involves analyzing flow errors, validation failures, and data inconsistencies when issues occur. Administrators review debug logs, flow error emails, and record history to identify root causes. Corrective actions may include updating flow logic, adjusting field-level security, refining decision conditions, or correcting data configurations. This structured approach ensures system stability, minimizes downtime, and supports continuous improvement of the support process.

12. Advantages & Disadvantages

12.1 Advantages

- Centralized management of all customer support cases in a single Salesforce app.
- Reduced manual effort through automated case classification, priority assignment, and SLA calculation.
- Faster, more consistent agent responses through AI-suggested replies via Agentforce.
- Improved SLA compliance visibility for support managers through real-time reports and dashboards.
- Strong data protection through role-based profiles, permission sets, and field-level security.
- A built-in feedback loop that measures and can improve AI suggestion quality over time.
- Built entirely on native Salesforce Admin and Flow features, keeping the solution low-cost and easy to maintain.

12.2 Disadvantages

- Requires an active Salesforce license and Agentforce/Einstein feature enablement, which may carry cost implications in a production org.
- Fully dependent on internet connectivity as a cloud-based platform.
- AI-suggested replies still require agent review before sending, so full automation of responses is intentionally not supported.
- Limited offline access, consistent with other cloud-native Salesforce applications.

13. Conclusion

The Salesforce AI-Powered Customer Support Ticketing System successfully demonstrates how intelligent automation and structured data modeling can improve the efficiency of customer support operations. By leveraging standard and custom Salesforce objects, record-triggered flows for case classification, priority assignment, and SLA tracking, and a well-defined security model, the system reduces manual effort and ensures timely issue resolution.

The inclusion of Agentforce-powered agent assistance and a structured feedback mechanism enhances agent productivity while enabling continuous improvement of the system's AI suggestions. Overall, the project delivers a scalable, maintainable, and real-world support solution, and provides strong hands-on experience with core Salesforce Admin, Flow automation, and Agentforce concepts — making it well suited for both practical implementation and interview demonstration.

14. Future Scope

- Extend Agentforce with multi-turn conversational case triage, allowing customers to interact directly with the agent before a human takes over.
- Introduce a knowledge base integration so AI-suggested replies can cite and link relevant help articles.
- Add escalation automation that reassigns or flags cases automatically when an SLA is close to breaching.
- Build a mobile-optimized agent experience for on-the-go case handling.
- Expand the AI feedback loop into a lightweight analytics dashboard tracking suggestion accuracy trends over time.
- Integrate omni-channel case creation (email-to-case, chat, and social) to broaden how customers can raise issues.

15. Appendix

Item	Details
Source	Salesforce Configuration — Objects, Flows, Validation Rules, Agentforce, Prompt Builder
Dataset	Sample Case, Contact, and Account records created during testing
Tools Used	Salesforce Developer Org, Flow Builder, Agentforce Agent Builder, Prompt Builder, Reports & Dashboards
Team	Durga Surya Teja (Individual Project)

Thank You